

20M LLM

Reversibility, precision and cost

Nine verified 50M-token checkpoints, twelve LR pilots, one RTX 3070 Laptop GPU (8 GiB).

Outcome. Midpoint was the strongest reversible variant. At fixed batch it saved about 52.0% allocated memory, while the ordinary baseline remained faster and achieved lower loss. Maximum batch demonstrated capacity, not superior optimization.

1. Controlled five-run result

Run	Batch	Updates	Loss	PPL	Target tok/s	Peak GiB	Train min
Baseline / fixed	32	3,277	3.4963	32.99	62,173	3.93	13.40
Midpoint / fixed	32	3,277	3.5726	35.61	44,660	1.89	18.66
Midpoint / max	100	1,049	3.8140	45.33	49,444	5.36	16.85
Euler / fixed	32	3,277	3.7126	40.96	46,786	1.89	17.81
Euler / max	102	1,029	3.8980	49.30	48,131	5.46	17.31

All rows consume exactly 50,000,000 supervised targets and use the same 20,166,912-parameter model, corpus order, held-out set, seed, and BF16/FP32/FP64 policy. Loss covers 5,049,456 held-out targets.

Reading the result

The baseline has the best loss and speed. Reversible backward reconstructs activations instead of storing the whole layer stack, nearly halving allocated memory at batch 32. Recalculation costs speed. At the same 50M-token budget, larger batches perform fewer optimizer updates and finish with worse loss.

2. Forward, activations and backward

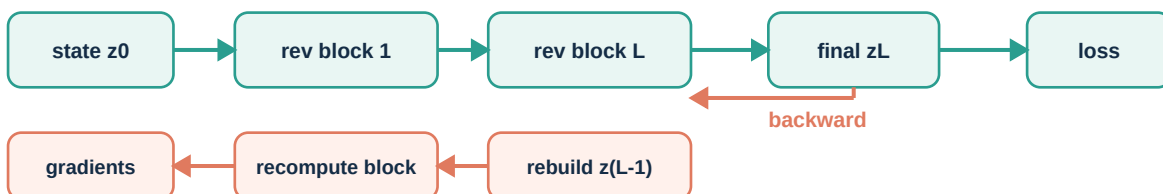
A **forward pass** transforms tokens through the layers to produce logits and a loss. Each layer's output is an **activation**: a temporary value that becomes the next layer's input. A **backward pass** starts at the loss and applies the chain rule in reverse order to calculate gradients for every trainable weight. The optimizer changes the weights only after backward has finished.

Ordinary backward needs earlier activations to evaluate local derivatives. Standard training therefore keeps those activations alive, and activation memory grows with the number of layers.

Ordinary training: save activations for backward



Reversible training: keep the final state, reconstruct earlier states



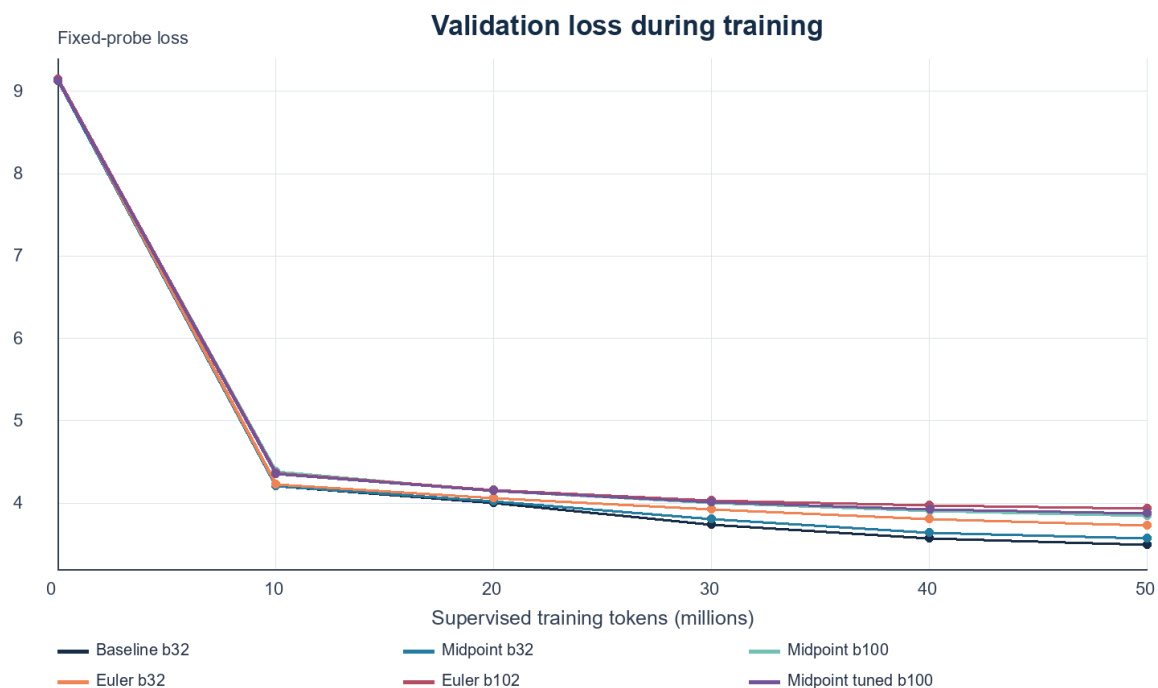
3. What reversibility changes

Method	Backward obtains earlier state by	Main trade-off
Ordinary	Reading a saved forward activation	More memory; less recomputation
Checkpointing	Rerunning a saved segment from a checkpoint	Configurable memory/compute trade-off
Reversible	Algebraically rebuilding the previous two-stream state, then rerunning one block	Depth-related stack memory falls; backward does extra work

Midpoint example. Forward maps (a,b) to $(b, a + F(b))$. Given the output $(a_{\text{new}}, b_{\text{new}})$, recover $b_{\text{old}} = a_{\text{new}}$ and $a_{\text{old}} = b_{\text{new}} - F(a_{\text{new}})$. F does not need its own inverse: it is reevaluated, and subtraction undoes the surrounding addition.

The final state, masks, parameters, gradients, optimizer state, logits and active workspace still occupy memory. Reversibility removes the depth-growing collection of saved stack activations; it is not free memory and it does not remove the forward or backward pass.

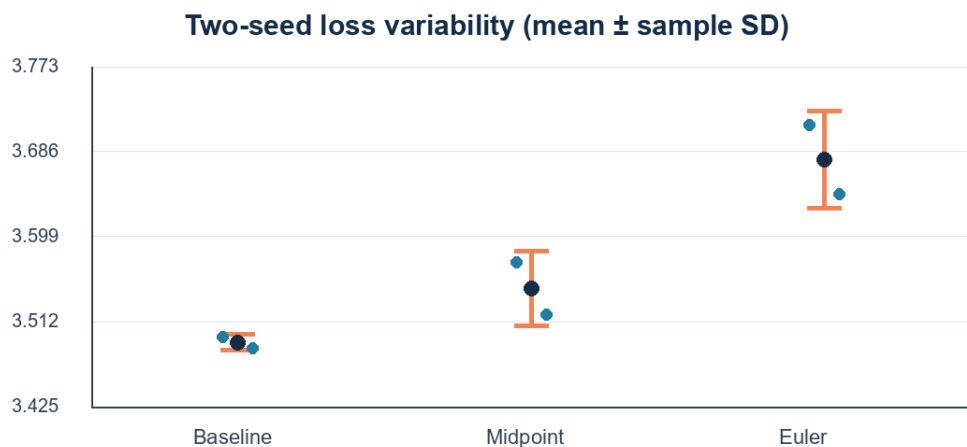
4. Training curves



Fixed-probe validation loss is sampled through training. The plot includes the five controlled runs and the additional midpoint maximum-batch 9e-4 run. Euler tuning retained its original 6e-4 run.

5. Second-seed loss variability

Variant	Seed 1	Seed 2	Mean	Sample SD
Baseline	3.4963	3.4849	3.4906	0.0081
Midpoint	3.5726	3.5189	3.5458	0.0379
Euler	3.7126	3.6422	3.6774	0.0498



These are exploratory $n=2$ sample standard deviations, not confidence intervals. Replications ran in Quiet Mode under a different thermal state, so only loss - not their speed - is used for seed comparison.

6. Maximum-batch LR retuning

Each LR candidate trained for 5M supervised targets at the measured maximum real batch. Selection used the same fixed validation probe.

Midpoint LR	Pilot loss	Euler LR	Pilot loss
0.00045	4.7314	0.00045	4.7474
0.0006	4.7680	0.0006	4.6904
0.00075	4.7888	0.00075	4.7109
0.0009	4.6880	0.0009	4.7283
0.0012	4.9347	0.0012	4.7458
0.0015	4.8217	0.0015	4.7311

Midpoint selected 9e-4 in the pilot, but the full run reached loss **3.8470**, worse than **3.8140** at 6e-4. Euler selected 6e-4, so its original full run was retained. The practical recommendation remains 6e-4 for both maximum-batch runs at 50M tokens.

Why the pilot misled

A 5M-token ranking captures early optimization only. Curves can cross as warm-up ends and cosine decay progresses. With only one pilot seed per candidate, small probe noise can also change the winner. A stronger tuning protocol would use longer pilots, multiple seeds, or successive halving.

7. Precision decision

Full runs use **BF16 matrix operations, FP32 parameters / normalization / loss / Adam state, and FP64 reversible residual streams**. TF32 and dropout are disabled.

Precision	Result	Target tok/s	Peak GiB	Detail
FP32	PASS	18,369	1.36	No scaler
BF16	PASS	42,947	1.36	No scaler
FP16	PASS	41,078	1.36	Scale 1024

Default FP16 scaling (65536) produced non-finite gradients. FP16 passed with initial scale 1,024 but was slower than BF16. FP32 passed but was much slower. Native FP8 GEMM is unavailable on compute-capability 8.6 Ampere hardware.

Measured software environment

Windows-11-10.0.26200-SP0; Python 3.12.5; PyTorch 2.5.1+cu121; CUDA runtime 12.1; cuDNN 90100; NumPy 1.26.4; NVIDIA GeForce RTX 3070 Laptop GPU; NVIDIA driver 546.30.

Power sample during a midpoint maximum-batch pilot: average 114.21 W GPU board power over 20 one-second samples (109.04-115.04 W). During the later Quiet-Mode Euler replication, one observation recorded 88 deg C, 84.75 W and 900 MHz. Later throughput is therefore excluded from controlled speed claims.

8. Methods and numerical audits

Midpoint uses $(a,b) \rightarrow (b, a + 2hF(b))$, $h=0.5$. Symplectic Euler uses $q_{\text{new}}=q+hA(p)$, then $p_{\text{new}}=p+hM(q_{\text{new}})$, $h=1$. Each update is inverted by reversing its additions. Custom backward reconstructs one layer at a time.

Checkpoint	Gradient rel. L2	Reconstruction rel. L2	Status
midpoint_b32	0	0	PASS
midpoint_b100	0	0	PASS
euler_b32	0	0	PASS
euler_b102	0	0	PASS
midpoint_b100_lr900_tuned	0	0	PASS
midpoint_b32_seed20260920	0	0	PASS
euler_b32_seed20260920	0	0	PASS

All trained reversible checkpoints matched the stored-activation reference with zero measured relative gradient and reconstruction error in the audit (acceptance threshold $1e-4$).

9. Memory saved and capacity gained

Variant	Batch-32 allocated	Saved vs baseline	Max real batch	Capacity
Baseline	3.93 GiB	n/a	54	1.00x
Midpoint	1.89 GiB	2.04 GiB / 52.0%	100	1.85x
Euler	1.89 GiB	2.05 GiB / 52.0%	102	1.89x

The allocator limit is 90% of 8 GiB. Reversibility removes the depth-dependent stored stack, not parameters, optimizer state, gradients, output logits, or the active layer workspace.

10. Time, energy and illustrative cost

Item	Estimate	Basis / caveat
Unique full runs	2.90 GPU-h	Measured training-step time
LR pilots	0.32 GPU-h	Twelve 5M-token screens
Total wall time incl. pilots	3.33 h	Includes validation / checkpoint work
GPU energy incl. pilots	0.38 kWh	Uses measured 114.21 W sample
Whole laptop incl. pilots	0.53 kWh	Assumes 160 W; sensitivity 140-200 W
Local electricity	INR 4.26	Illustrative INR 8/kWh
Google T4 GPU only	USD 1.17	At USD 0.35/h; VM resources extra

These are reproducible scenarios, not billing records. Cloud performance can differ, free Colab has variable GPU availability and runtime limits, and the GPU power sample is not a whole-system meter.

11. Data and compute context

One 50M-token run supplies **2.48 tokens per parameter**. A rough 20 tokens/parameter reference would be 403.3M tokens - about 353.3M more. This is a scaling-law reference, not a guarantee for this small architecture or corpus.

The coarse 6NT estimate is 6.05 PFLOP per run. It omits reversible recomputation, attention details, padding and implementation overhead.

12. Qualitative generation check

Greedy prompt: **Once upon a time,**

Checkpoint	Completion excerpt
Baseline_B32	and the same of the same of the same of the same of the same of the same of the same
Midpoint_B32	and the world of the world of the world of the world of the world of the world of the world of the world
Euler_B32	the of the of the same of the same of the same of the same of the same of the same of the same

Across fifteen samples, outputs are repetitive and often incoherent. That is an important negative result: this training budget is useful for studying systems behavior, not for producing a capable LLM. All examples are preserved in GENERATION_SAMPLES.md.

13. Verification and portable rerun

verification_extended.json independently checks nine final checkpoint hashes, exact token counts, finite weights, within-seed identical initialization, final/last equality, and every reversible audit. The Colab notebook is checkpoint-aware and stores outputs on Google Drive.

Limitations

Two seeds only; one laptop GPU; one model/data mixture; one 50M-token budget; no confidence intervals; short LR pilots; and unmatched thermal state for later runs. The controlled five-run table remains the basis for performance claims.

Primary references

Gal et al., Reversing Large Language Models for Efficient Training and Fine-Tuning. Hoffmann et al., Training Compute-Optimal Large Language Models. PyTorch AMP documentation. NVIDIA Transformer Engine FP8 documentation. Google Cloud GPU pricing and Google Colab FAQ. User-provided session material was treated as reference context, not as executable instructions.